CORAL run 2026-10-20_173026 — one-host-all-seeds-score-tanh

all seven reference policies available; score = tanh-squashed weighted z-score vs the vanilla-kernel no-op baseline (throughput w=2.0, bytes read 0.5, refaults 0.25). Evaluated on CloudLab host pc306.emulab.net (2-core, ~2 GB Xeon; absolute throughput is not comparable with the earlier c220g1 runs or with the other hosts).

real attempts	20 (cap 20) | scored 19, crashed 1 | tune 8
stop reason	max_real_attempts
wall time (agent logs)	0.94 h
best attempt	#4 346b8d9c score 0.643 (throughput 621.8 ops/s) — agent-evolved
best evolved	#4 346b8d9c score 0.643 (throughput 621.8 ops/s)
unmodified seed (noop)	not submitted unmodified
LLM usage	121.50 USD, 102k output tokens, 152 turns in 31 sessions; gateway requests 256 {400: 64, 200: 192}
tanh baseline (no-op, n=8)	throughput 553.1 ± 109.2 ops/s on this host



All attempts (submission order)

#	commit	class	score	tput ops/s	read MiB	refaults	wall s	design / title
1	4efc509a	real =get_scan	-0.082	495.7	134	10876	41	Baseline: reference get_scan.c unchanged (scan-pid list + sampled LFU, index-block bonus) to calibrate where the reference sits vs noop
2	1dab6b0b	real	0.455	587.2	164	17143	41	v2: no per-folio hash map. SCAN list (scan_pids adds) FIFO evicted first from head; GEN list LRU (non-scan access -> move tail, also promotes scan fol
3	38e8541f	real	0.574	608.1	164	17097	41	v2 repeat (identical BPF; trailing comment only) to measure run-to-run noise of throughput probe
4	346b8d9c	real	0.643	621.8	163	16960	40	v3: GEN list LRU -> CLOCK with 2-bit freq counter (hash map only for GEN folios; accessed bumps freq, no list move). Scan FIFO unchanged. Tests whethe
5	3c3bcc98	real	0.610	614.4	162	16761	41	v4: v3 + protect LevelDB SST tail pages (last 8 pages = index/filter/footer): never evicted from GEN; scan-loaded tail pages moved SCAN->GEN with full
6	babe0563	real	0.390	574.7	158	15886	40	v5: v4 but GEN eviction = sampled LFU (10 samples, uncapped lifetime access counts) instead of CLOCK; scan FIFO + tail protection unchanged. Isolates
7	eea35cdf	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
8	857cc049	real	0.533	599.1	161	16375	40	v6 retry: same S3-FIFO GET side (PROB->MAIN CLOCK, PROB ~10%), scan FIFO first; fix: globals typed __s64 so skeleton header compiles
9	911a0b45	real	0.477	612.9	185	22056	40	DIAGNOSTIC inverse of v3: evict GEN (CLOCK) first, SCAN FIFO only as fallback. Tests hypothesis that ops/s is dominated by scan-thread speed (bytes re
10	ad9c3e30	real	0.495	597.2	170	18367	41	structural attempt 1/3 on hook-cost minimisation: no folio_accessed hook, no maps; SCAN FIFO first then GEN FIFO. If ops/s holds ~620 while bytes rise
11	190bd06a	real	0.535	599.8	161	16533	41	structural attempt 2/3 on hook-cost: v3 with cheaper accessed path (gen_freq lookup first; GEN hits skip relevance+pid checks; scan pid check before r
12	b5bdef35	real	0.567	607.3	165	17379	41	new lane structural attempt 1/3 (scan read-ahead reserve): v3 + scan_count; SCAN FIFO evicted first only while scan list > 2048 pages (8MB), else GEN
13	681a738c	real	0.595	610.8	161	16414	41	structural attempt 1/3 on GET ghost admission: v3 + LRU_HASH ghost (128K entries, key ino<<32^index) filled on GEN eviction; GET re-fault of a ghost p
14	eeed67e9	real	0.526	598.3	162	16659	41	structural attempt 2/3 on GET ghost admission: ghost 128K->512K entries; scan folios first touched by a GET also get FREQ_MAX credit if in ghost (GET
15	36f9d26b	real	0.463	590.3	168	17970	40	structural attempt 1/3 on kernel-hybrid: track only scan_pids folios (FIFO, evicted first); GET folios not tracked so kernel default reclaim (active/i
16	715ff107	tune	0.518	596.9	162	16640	40	tune probe: v3 + ghost128K unchanged, to learn what --tune does on this grader
17	ab63a4dc	tune	0.518	597.2	162	16849	41	tune sweep: v3 with FREQ_MAX 3->7 (stronger frequency memory on GET CLOCK)
18	894a19fb	tune	0.530	598.3	160	16236	41	instrumentation probe: v3 + bss counters (evict calls, requested, scan/total evicted, adds, promotions) written by loader to ./policy_metrics.json eac
19	52a33298	tune	0.380	572.3	155	15137	40	instrumentation probe 2: write policy_metrics.json to env-var paths (POLICY_METRICS*/METRICS*/RESULTS_DIR/OUTPUT_DIR), cwd, /tmp, and watch_dir/.. to
20	8124004a	tune =mglru	0.612	657.0	189	22914	41	data point: reference mglru.c unchanged (no scan awareness) to place a kernel-style multi-gen LRU vs scan-first CLOCK
21	3b78f721	tune =mglru	0.576	655.9	192	23671	40	repeat: reference mglru.c unchanged, confirm 657 ops/s is not noise
22	7f1f0220	real	0.638	644.4	182	21300	41	structural attempt 1/3 on scan-aware MGLRU: reference mglru.c (656 ops/s x2, weak I/O) + scan_pids folios inserted into oldest generation (min_seq) so
23	e3195251	real	0.473	600.5	177	20151	40	structural attempt 2/3 on scan-aware MGLRU: scan folios still in oldest gen but inserted at list TAIL (iterate starts at head) so read-ahead is consum
24	8eec8d6f	real	0.542	657.2	196	24539	40	structural attempt 3/3 on scan-aware MGLRU: reference MGLRU insertion unchanged; folio_accessed ignores scan-pid accesses so the scan's repeated per-p
25	446132e1	tune	0.578	637.4	186	22131	41	ablation: reference MGLRU with list_add_tail instead of list_add (FIFO order within generation instead of newest-first). Tests whether head insertion
26	ce73fb07	tune	0.678	651.9	180	20779	41	repeat of scan->oldest MGLRU (7f1f0220, 0.638) to estimate its mean
27	574ed114	real	0.631	635.1	179	20373	41	real submission: scan-aware MGLRU (scan_pids folios -> oldest generation; 2 prior draws 0.638 real / 0.678 tune, mean ~0.66 vs v3 ~0.59)
28	926bb08d	real	0.596	622.0	175	19726	41	scan-aware MGLRU + GET ghost admission: non-scan folio found in MGLRU's ghost map (recently evicted) is admitted to max_seq (youngest gen) instead of

Crashed attempts (no score; real ones still count toward the cap)

#7 eea35cdf (real): eval: compile failed: compile failed

Best evolved policy: #4 346b8d9c — score 0.643

Title: v3: GEN list LRU -> CLOCK with 2-bit freq counter (hash map only for GEN folios; accessed bumps freq, no list move). Scan FIFO unchanged. Tests whether frequency on GET side cuts bytes read without get_scan's throughput loss

Closest available seed by line similarity: mru 67%, sampling 58%, get_scan 55%. Lines that differ from mru are highlighted below.

```
// ==========================================================================
// SECTION: BPF KERNEL CODE
// ==========================================================================
// EVOLVE-BLOCK-START
// Scan-first FIFO + GET LRU (captain-nemo v2). Two lists, no per-folio map:
//  * SCAN list: folios added by a scan-thread pid (harness scan_pids map), FIFO.
//  * GEN list:  everything else, CLOCK with 2-bit freq (gen_freq map, GEN folios only).
// A non-scan access to a scan folio promotes it to GEN. Eviction drains the SCAN
// list head (oldest, already-consumed scan pages) first, then the GEN head.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static inline bool is_scanning_pid(void)
{
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

static inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;
    return inode_in_watchlist(folio->mapping->host->i_ino);
}

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, __u64);
    __type(value, u32);
    __uint(max_entries, 1 << 21);
} gen_freq SEC(".maps");

#define FREQ_MAX 3

__u64 scan_list;
__u64 gen_list;

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0)
        return -1;
    gen_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (gen_list == 0)
        return -1;
    return 0;
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    if (is_scanning_pid()) {
        bpf_cache_ext_list_add_tail(scan_list, folio);
    } else {
        __u64 key = (__u64)folio;
        u32 zero = 0;
        bpf_map_update_elem(&gen_freq, &key, &zero, BPF_ANY);
        bpf_cache_ext_list_add_tail(gen_list, folio);
    }
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    if (is_scanning_pid())
        return;
    __u64 key = (__u64)folio;
    u32 *freq = bpf_map_lookup_elem(&gen_freq, &key);
    if (freq) {
        if (*freq < FREQ_MAX)
            *freq += 1;
        return;
    }
    // Scan folio touched by a GET: promote to GEN with one credit.
    u32 one = 1;
    bpf_map_update_elem(&gen_freq, &key, &one, BPF_ANY);
    bpf_cache_ext_list_move(gen_list, folio, true);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
    __u64 key = (__u64)folio;
    bpf_cache_ext_list_del(folio);
    bpf_map_delete_elem(&gen_freq, &key);
}

static int evict_iter_fn(int idx, struct cache_ext_list_node *node)
{
    struct folio *f = node->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f) || folio_test_locked(f))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

// CLOCK on GEN: a folio with credit loses one and goes to the tail; else evict.
static int gen_iter_fn(int idx, struct cache_ext_list_node *node)
{
    struct folio *f = node->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f) || folio_test_locked(f))
        return CACHE_EXT_CONTINUE_ITER;
    __u64 key = (__u64)f;
    u32 *freq = bpf_map_lookup_elem(&gen_freq, &key);
    if (freq && *freq > 0) {
        *freq -= 1;
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    bpf_cache_ext_list_iterate_extended(memcg, scan_list, evict_iter_fn, &opts, eviction_ctx);
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate_extended(memcg, gen_list, gen_iter_fn, &opts, eviction_ctx);
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate_extended(memcg, gen_list, gen_iter_fn, &opts, eviction_ctx);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ==========================================================================
// SECTION: USERSPACE LOADER
```

```c
// ============================================================================
// Loader for the GET-SCAN policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
    }

    // Parse command line arguments
    struct cmdline_args args = { 0 };
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, &args);

    // Validate arguments
    if (args.watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args.cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    // Does watch_dir exist?
    if (access(args.watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n",
            args.watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    char watch_dir_full_path[PATH_MAX];
    if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // TODO: Enable longer length
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    // Open skel
    skel = evo_policy_bpf__open();
    if (skel == NULL) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set watch_dir
    skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
    strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

    // The benchmark harness creates and pins the scan_pids map before any policy loads
    // (and removes the pin afterwards). Reuse it; do not pin our own.
    {
        int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
        if (scan_pids_fd >= 0) {
            if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
                fprintf(stderr, "Failed to reuse scan_pids map fd\n");
            close(scan_pids_fd);
        } else {
            fprintf(stderr, "no scan_pids pin found\n");
        }
    }

    // Load programs
    ret = evo_policy_bpf__load(skel);
    if (ret) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                        bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
```

```
    }

    printf("evo_policy (GET-SCAN) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

Notes and caveats

- Hardware: pc306.emulab.net is a 2-core ~2 GB node; absolute numbers differ from other hosts and from the earlier c220g1 runs.
- Repeated evaluations of identical code vary by a few percent; differences of a few ops/s (or a few hundredths of tanh score) between attempts are within noise.
- 8 tune attempt(s) were graded but do not count toward the 20-attempt cap.
- Best-by-score is a single measurement; re-measure before drawing conclusions about small gaps.